

Кафедра ІПЗ

Лабораторна робота №7

з теми «Побудова базового автоматизаційного
рішення»

Виконала:

ст. гр. ІП-22-1

Чемна (Микитин) О. І.

Перевірив:

Храбатин Р. І.

Мета: Зрозуміти та навчитися будувати прості базові автоматизовані рішення.

Короткі теоретичні відомості:

Автоматизаційне рішення — це організований набір тестів, допоміжних функцій, налаштувань, тестових даних і скриптів запуску, який дозволяє системно перевіряти програмний продукт.

На відміну від окремих тестових файлів, автоматизаційне рішення має структуру, у якій різні типи тестів розділені за призначенням. Це спрощує підтримку, повторне використання коду та подальше розширення тестового покриття.

Якісна автоматизація починається з архітектури. Важливо не просто написати багато тестів, а зробити їх зрозумілими, стабільними та зручними для підтримки. Для цього використовуються принципи DRY, KISS, SOLID і POM, які допомагають уникати дублювання коду, не ускладнювати рішення без потреби та розділяти відповідальність між різними частинами проєкту.

У межах автоматизаційного рішення можуть бути різні рівні тестування: unit, API, UI та E2E. Unit-тести перевіряють окремі функції, API-тести перевіряють взаємодію з сервером, UI-тести перевіряють графічний інтерфейс, а E2E-тести перевіряють повні користувацькі сценарії. У цій роботі ці типи тестів були об'єднані в один структурований проєкт, який можна запускати повністю або окремими командами.

Завдання:

Завдання перше:

Відносно обраного в попередній роботі сайту розробити автоматизаційне рішення яке включатиме у себе unit, ui, api, та e2e тестування. Структура рішення може бути довільна, але разом з тим логічна. Стосовно тестів: e2e можна скопіювати з попередньої роботи, оскільки це найгроміздкіша частина і на її розробку пішло б багато часу. Для решти тестів оберіть одну сторінку і пропрацюйте над її тестуванням.

Щодо обсягу кожного з видів – хоча б по п'ять тестів для кожного типу. Для функцій, які ви розробите в межах оптимізації тестів створіть компонентні тести.

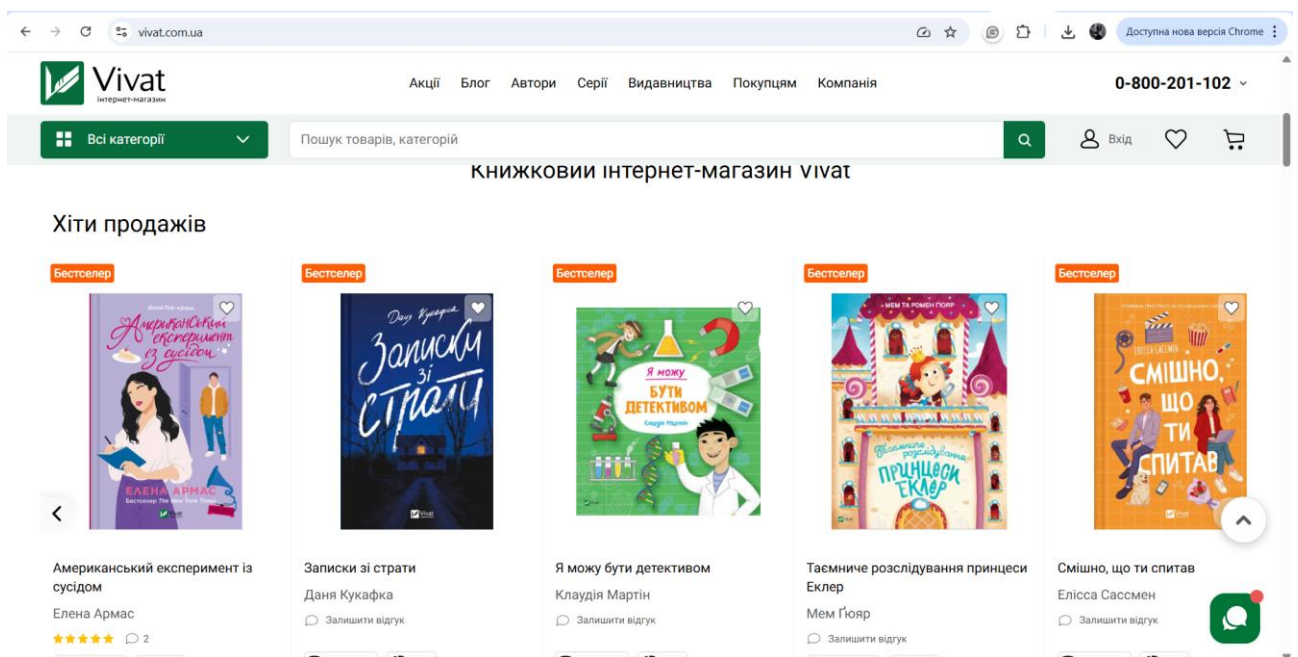
При роботі послуговуйтеся підходами KISS та DRY.

Завдання друге:

Завантажити код на власний/навчальний репозиторій GitHub, або інший схожий ресурс та поділитися посиланням.

Виконання:

Для виконання роботи я обрала сайт Видавництва Віват:



Цей сайт я використовувала у попередніх роботах, тому він добре підійшов для побудови базового автоматизаційного рішення. Для API-тестування я використала сервіс JSONPlaceholder, оскільки сайт Віват не надає зручного відкритого навчального API для перевірки методів GET, POST, PUT і DELETE.

Спочатку я створила новий каталог для автоматизаційного рішення:

```
oksanka@debian:~/Documents/automated_testing$ mkdir automation-solution
oksanka@debian:~/Documents/automated_testing$ cd automation-solution
```

Після цього я ініціалізувала Node.js-проект.

```

• oksanka@debian:~/Documents/automated_testing/automation-solution$ npm init -y
Wrote to /home/oksanka/Documents/automated_testing/automation-solution/package.json:

{
  "name": "automation-solution",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

○ oksanka@debian:~/Documents/automated_testing/automation-solution$

```

Далі я встановила необхідні залежності для тестування:

```

• oksanka@debian:~/Documents/automated_testing/automation-solution$ npm install axios

added 23 packages, and audited 24 packages in 2s

6 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
• oksanka@debian:~/Documents/automated_testing/automation-solution$ npm install --save-dev jest puppeteer
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache
e if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and p
owerful.
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulne
rabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at
exorbitant rates) by contacting i@izs.me
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulne
rabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (a
t exorbitant rates) by contacting i@izs.me

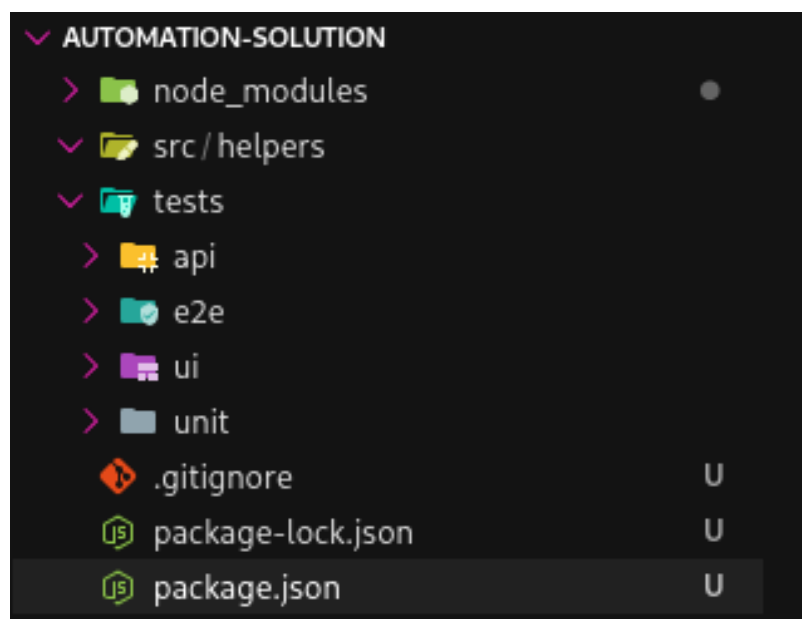
added 358 packages, and audited 382 packages in 19s

54 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
○ oksanka@debian:~/Documents/automated_testing/automation-solution$

```

Після цього я створила структуру проєкту:



Далі я налаштувала файл `package.json`, щоб можна було запускати всі тести разом або окремо за типами:

```
6   "scripts": {
7     "test": "jest",
8     "test:unit": "jest tests/unit",
9     "test:api": "jest tests/api",
10    "test:ui": "jest tests/ui",
11    "test:e2e": "jest tests/e2e"
12  },
```

Після цього я створила допоміжні функції, які використовуються для обробки тексту. Ці функції можна застосовувати під час тестування сторінок сайту, наприклад для нормалізації тексту, перевірки наявності слів або очищення зайвих пробілів.

У файл `src/helpers/textHelpers.js` я додала такий код:

```
textHelpers.js X Search automation-solution — textHelpers.js - automation-solution - V
src > helpers > JS textHelpers.js > ...
1  function normalizeText(text) {
2    | return text.toLowerCase().trim();
3  }
4
5  function hasText(source, expectedText) {
6    | return normalizeText(source).includes(normalizeText(expectedText));
7  }
8
9  function removeExtraSpaces(text) {
10   | return text.replace(/\s+/g, " ").trim();
11 }
12
13 function isEmptyText(text) {
14   | return typeof text === "string" && text.trim().length > 0;
15 }
16
17 function countWords(text) {
18   | return removeExtraSpaces(text).split(" ").filter(Boolean).length;
19 }
20
21 module.exports = {
22   normalizeText,
23   hasText,
24   removeExtraSpaces,
25   isEmptyText,
26   countWords,
27 };
```

Після створення допоміжних функцій я написала unit-тести для їх перевірки. У файл `tests/unit/textHelpers.test.js` я додала такий код:

```
textHelpers.js  textHelpers.test.js X
tests > unit > textHelpers.test.js > ...
1  const {
2    normalizeText,
3    hasText,
4    removeExtraSpaces,
5    isEmptyText,
6    countWords,
7  } = require("../src/helpers/textHelpers");
8
9  describe("Text helper unit tests", () => {
10    test("normalizeText should convert text to lowercase and trim spaces", () => {
11      expect(normalizeText("  VIVAT Books  ")).toBe("vivat books");
12    });
13
14    test("hasText should return true when source contains expected text", () => {
15      expect(hasText("Видавництво Віват", "віват")).toBe(true);
16    });
17
18    test("removeExtraSpaces should remove duplicated spaces", () => {
19      expect(removeExtraSpaces("Книга українською мовою")).toBe(
20        "Книга українською мовою",
21      );
22    });
23
24    test("isEmptyText should return true for non-empty string", () => {
25      expect(isEmptyText("Книжковий каталог")).toBe(true);
26    });
27
28    test("countWords should count words in text", () => {
29      expect(countWords("Видавництво Віват книги")).toBe(3);
30    });
31  });
```

Далі я створила API-тести. Для цього я використала сервіс JSONPlaceholder. У файл tests/api/api.test.js я додала такий код:

```
textHelpers.js U  textHelpers.test.js U  api.test.js U X
tests > api > api.test.js > describe("API tests") callback > test("DELETE /posts/1 should return status code 200") callback
1  const axios = require("axios"); 72k (gzipped: 26.5k)
2
3  const BASE_URL = "https://jsonplaceholder.typicode.com";
4
5  describe("API tests", () => {
6    test("GET /posts should return status code 200", async () => {
7      const response = await axios.get(`${BASE_URL}/posts`);
8
9      expect(response.status).toBe(200);
10    });
11
12    test("GET /posts should return an array", async () => {
13      const response = await axios.get(`${BASE_URL}/posts`);
14
15      expect(Array.isArray(response.data)).toBe(true);
16    });
17
18    test("GET /posts/1 should return post with id 1", async () => {
19      const response = await axios.get(`${BASE_URL}/posts/1`);
20
21      expect(response.data.id).toBe(1);
22    });
23  });
```

```

22 //
23
24 test("POST /posts should create a new post", async () => {
25   const response = await axios.post(`${BASE_URL}/posts`, {
26     title: "Vivat test post",
27     body: "Automation solution API test",
28     userId: 1,
29   });
30
31   expect(response.status).toBe(201);
32   expect(response.data.title).toBe("Vivat test post");
33 });
34
35 test("DELETE /posts/1 should return status code 200", async () => {
36   const response = await axios.delete(`${BASE_URL}/posts/1`);
37
38   expect(response.status).toBe(200);
39 });
40
41

```

Після цього я створила UI-тести для сайту Видавництва Віват. У файл tests/ui/ui.test.js я додала такий код:

```

textHelpers.js U  textHelpers.test.js U  api.test.js U  ui.test.js U X
tests > ui > ui.test.js > ...
1  const puppeteer = require("puppeteer");
2
3  jest.setTimeout(90000);
4
5  const BASE_URL = "https://vivat.com.ua/";
6  const BOOKS_URL = "https://vivat.com.ua/category/knyhy/";
7
8  describe("Vivat UI tests", () => {
9    let browser;
10    let page;
11
12    beforeEach(async () => {
13      browser = await puppeteer.launch({
14        headless: true,
15        args: ["--no-sandbox", "--disable-setuid-sandbox"],
16      });
17
18      page = await browser.newPage();
19      await page.setViewport({ width: 1366, height: 768 });
20    });
21
22    afterEach(async () => {
23      await browser.close();
24    });

```

```

26   test("should open Vivat home page", async () => {
27     await page.goto(BASE_URL, { waitUntil: "networkidle2" });
28
29     expect(page.url()).toContain("vivat.com.ua");
30   });
31
32   test("home page should have title with Vivat", async () => {
33     await page.goto(BASE_URL, { waitUntil: "networkidle2" });
34
35     const title = await page.title();
36
37     expect(title.toLowerCase()).toContain("vivat");
38   });

```

```

39
40 test("home page should contain links", async () => {
41     await page.goto(BASE_URL, { waitUntil: "networkidle2" });
42
43     const linksCount = await page.$$eval("a", (links) => links.length);
44
45     expect(linksCount).toBeGreaterThan(0);
46 });
47
48 test("books category page should open", async () => {
49     await page.goto(BOOKS_URL, { waitUntil: "networkidle2" });
50
51     expect(page.url()).toContain("/category/knyhy");
52 });
53
54 test("books category page should contain book-related text", async () => {
55     await page.goto(BOOKS_URL, { waitUntil: "networkidle2" });
56
57     const bodyText = await page.$eval("body", (body) => body.innerText);
58
59     expect(bodyText.toLowerCase()).toContain("книг");
60 });
61 });
62

```

Далі я створила E2E-тести для перевірки користувацьких сценаріїв. У файл tests/e2e/e2e.test.js я додала такий код:

```

js textHelpers.js U  textHelpers.test.js U  api.test.js U  ui.test.js U  e2e.test.js U X
tests > e2e > e2e.test.js > ...
1  const puppeteer = require("puppeteer");
2
3  jest.setTimeout(120000);
4
5  const BASE_URL = "https://vivat.com.ua/";
6  const BOOKS_URL = "https://vivat.com.ua/category/knyhy/";
7
8  async function getBodyText(page) {
9      return page.$eval("body", (body) => body.innerText);
10 }
11
12 async function getLinksCount(page) {
13     return page.$$eval("a", (links) => links.length);
14 }
15
16 async function typeIntoSearch(page, searchText) {
17     const selectors = [
18         "input[type='search']",
19         "input[name='q']",
20         "input[name='search']",
21         "input[placeholder*='Пошук']",
22         "input[placeholder*='пошук']",
23     ];
24
25     for (const selector of selectors) {
26         const input = await page.$(selector);
27
28         if (input) {
29             await input.click({ clickCount: 3 });
30             await input.type(searchText);
31             await page.keyboard.press("Enter");
32             return true;
33         }
34     }
35
36     return false;
37 }
38

```



```

39  async function clickFirstVisibleLinkWithText(page, textVariants) {
40      return page.evaluate((variants) => {
41          const normalizedVariants = variants.map((item) => item.toLowerCase());
42          const links = Array.from(document.querySelectorAll("a"));
43
44          const target = links.find((link) => {
45              const text = link.innerText.toLowerCase().trim();
46
47              return (
48                  link.href &&
49                  text.length > 0 &&
50                  normalizedVariants.some((variant) => text.includes(variant))
51              );
52          });
53
54          if (target) {
55              target.click();
56              return true;
57          }
58
59          return false;
60      }, textVariants);
61  }
62

```

```

63  describe("Vivat E2E tests", () => {
64      let browser;
65      let page;
66
67      beforeEach(async () => {
68          browser = await puppeteer.launch({
69              headless: true,
70              args: ["--no-sandbox", "--disable-setuid-sandbox"],
71          });
72
73          page = await browser.newPage();
74          await page.setViewport({ width: 1366, height: 768 });
75      });
76
77      afterEach(async () => {
78          await browser.close();
79      });
80
81      test("user can open home page and see content", async () => {
82          await page.goto(BASE_URL, { waitUntil: "networkidle2" });
83
84          const bodyText = await getBodyText(page);
85
86          expect(bodyText.length).toBeGreaterThan(100);
87      });
88

```

```

89      test("user can open books category from direct link", async () => {
90          await page.goto(BOOKS_URL, { waitUntil: "networkidle2" });
91
92          const bodyText = await getBodyText(page);
93
94          expect(bodyText.toLowerCase()).toContain("книг");
95      });
96
97      test("user can search for a book", async () => {
98          await page.goto(BASE_URL, { waitUntil: "networkidle2" });
99
100         const searchFound = await typeIntoSearch(page, "книга");
101
102         expect(searchFound).toBe(true);
103     });

```

```

104     await page
105       .waitForNavigation({ waitUntil: "networkidle2" })
106       .catch(() => null);
107
108     const bodyText = await getBodyText(page);
109
110     expect(bodyText.toLowerCase()).toContain("книг");
111   });
112
113   test("user can navigate from category to book-related page", async () => {
114     await page.goto(BOOKS_URL, { waitUntil: "networkidle2" });
115
116     const clicked = await clickFirstVisibleLinkWithText(page, [
117       "книга",
118       "роман",
119       "видання",
120       "комікс",
121       "манґа",
122     ]);
123
124     expect(clicked).toBe(true);
125
126     await page
127       .waitForNavigation({ waitUntil: "networkidle2" })
128       .catch(() => null);
129
130     expect(page.url()).toContain("vivat.com.ua");
131   });
132

```

```

133   test("user can see many navigation links after opening category", async () => {
134     await page.goto(BOOKS_URL, { waitUntil: "networkidle2" });
135
136     const linksCount = await getLinksCount(page);
137
138     expect(linksCount).toBeGreaterThan(5);
139   });
140 });
141

```

Після написання всіх тестів я послідовно запустила кожен тип тестування окремо:

```

oksanka@debian:~/Documents/automated_testing/automation-solution$ npm run test:unit

> automation-solution@1.0.0 test:unit
> jest tests/unit

PASS tests/unit/textHelpers.test.js
Text helper unit tests
  ✓ normalizeText should convert text to lowercase and trim spaces (2 ms)
  ✓ hasText should return true when source contains expected text
  ✓ removeExtraSpaces should remove duplicated spaces (1 ms)
  ✓ isEmptyText should return true for non-empty string
  ✓ countWords should count words in text

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
Snapshots:   0 total
Time:        0.214 s
Ran all test suites matching tests/unit.
oksanka@debian:~/Documents/automated_testing/automation-solution$

```

```
● oksanka@debian:~/Documents/automated_testing/automation-solution$ npm run test:api
```

```
> automation-solution@1.0.0 test:api
> jest tests/api
```

```
PASS tests/api/api.test.js
  API tests
    ✓ GET /posts should return status code 200 (256 ms)
    ✓ GET /posts should return an array (60 ms)
    ✓ GET /posts/1 should return post with id 1 (54 ms)
    ✓ POST /posts should create a new post (147 ms)
    ✓ DELETE /posts/1 should return status code 200 (143 ms)
```

```
Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
Snapshots:   0 total
Time:        0.901 s
Ran all test suites matching tests/api.
```

```
● oksanka@debian:~/Documents/automated_testing/automation-solution$
```

```
● oksanka@debian:~/Documents/automated_testing/automation-solution$ npm run test:ui
```

```
> automation-solution@1.0.0 test:ui
> jest tests/ui
```

```
PASS tests/ui/ui.test.js (31.057 s)
  Vivat UI tests
    ✓ should open Vivat home page (6377 ms)
    ✓ home page should have title with Vivat (6249 ms)
    ✓ home page should contain links (5137 ms)
    ✓ books category page should open (8370 ms)
    ✓ books category page should contain book-related text (4658 ms)
```

```
Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
Snapshots:   0 total
Time:        31.097 s
Ran all test suites matching tests/ui.
```

```
● oksanka@debian:~/Documents/automated_testing/automation-solution$
```

```
● oksanka@debian:~/Documents/automated_testing/automation-solution$ npm run test:e2e
```

```
> automation-solution@1.0.0 test:e2e
> jest tests/e2e
```

```
PASS tests/e2e/e2e.test.js (63.817 s)
  Vivat E2E tests
    ✓ user can open home page and see content (7102 ms)
    ✓ user can open books category from direct link (5219 ms)
    ✓ user can search for a book (37539 ms)
    ✓ user can navigate from category to book-related page (5930 ms)
    ✓ user can see many navigation links after opening category (7743 ms)
```

```
Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
Snapshots:   0 total
Time:        63.864 s
Ran all test suites matching tests/e2e.
```

```
● oksanka@debian:~/Documents/automated_testing/automation-solution$
```

Також я запустила всі тести однією командою:

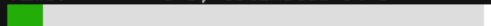
```
● oksanka@debian:~/Documents/automated_testing/automation-solution$ npm test
```

```
> automation-solution@1.0.0 test
> jest
```

```
PASS tests/api/api.test.js
PASS tests/unit/textHelpers.test.js
```

```
RUNS tests/e2e/e2e.test.js
RUNS tests/ui/ui.test.js
```

```
Test Suites: 2 passed, 2 of 4 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        6 s, estimated 64 s
```



```
oksanka@debian:~/Documents/automated_testing/automation-solution$ npm test

> automation-solution@1.0.0 test
> jest

PASS tests/api/api.test.js
PASS tests/unit/textHelpers.test.js
PASS tests/ui/ui.test.js (33.575 s)
PASS tests/e2e/e2e.test.js (60.974 s)

Test Suites: 4 passed, 4 total
Tests: 20 passed, 20 total
Snapshots: 0 total
Time: 61.177 s, estimated 64 s
Ran all test suites.
oksanka@debian:~/Documents/automated_testing/automation-solution$
```

У результаті я отримала автоматизаційне рішення, яке включає чотири типи тестів.

Тип тестування	Файл	Кількість тестів
Unit	tests/unit/textHelpers.test.js	5
API	tests/api/api.test.js	5
UI	tests/ui/ui.test.js	5
E2E	tests/e2e/e2e.test.js	5

Після успішного запуску тестів я підготувала проєкт до завантаження на GitHub. У мене раніше уже був ініціалізований Git-репозиторій для виконання цих робіт.

Тому першим ділом я перевірила стан файлів:

```
oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Невідстежувані файли:
  (скористайтесь "git add <файл>...", щоб додати до майбутнього коміту)
  automation-solution/

нічого не додано до коміту, але є невідстежувані файли (скористайтесь "git add" для відстежування)
oksanka@debian:~/Documents/automated_testing$ git add
```

Далі я додала всі файли до індексації Git:

```
oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Зміни, додані до майбутнього коміту:
  (використовуйте "git restore --staged <файл>...", щоб видалити з індексу)
  новий файл: automation-solution/.gitignore
  новий файл: automation-solution/package-lock.json
  новий файл: automation-solution/package.json
  новий файл: automation-solution/src/helpers/textHelpers.js
  новий файл: automation-solution/tests/api/api.test.js
  новий файл: automation-solution/tests/e2e/e2e.test.js
  новий файл: automation-solution/tests/ui/ui.test.js
  новий файл: automation-solution/tests/unit/textHelpers.test.js
oksanka@debian:~/Documents/automated_testing$
```

Після цього я створила коміт:

```
oksanka@debian:~/Documents/automated_testing$ git commit -m "Build basic automation solution for lab 7"
[main clcf2b8] Build basic automation solution for lab 7
8 files changed, 5791 insertions(+)
create mode 100644 automation-solution/.gitignore
create mode 100644 automation-solution/package-lock.json
create mode 100644 automation-solution/package.json
create mode 100644 automation-solution/src/helpers/textHelpers.js
create mode 100644 automation-solution/tests/api/api.test.js
create mode 100644 automation-solution/tests/e2e/e2e.test.js
create mode 100644 automation-solution/tests/ui/ui.test.js
create mode 100644 automation-solution/tests/unit/textHelpers.test.js
oksanka@debian:~/Documents/automated_testing$
```

Потім я завантажила проєкт на GitHub:

```
oksanka@debian:~/Documents/automated_testing$ git push -u origin main
Username for 'https://github.com': OksanaMykytyn
Password for 'https://OksanaMykytyn@github.com':
Перерахування об'єктів: 19, готово.
Підрахунок об'єктів: 100% (19/19), готово.
Дельта компресія з використанням до 4 потоків
Компресія об'єктів: 100% (11/11), готово.
Запис об'єктів: 100% (18/18), 48.80 KiB | 48.80 MiB/c, готово.
Всього 18 (дельта 1), повторно використано 17 (дельта 0), повторно використано пакунків 0 (з 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/OksanaMykytyn/automated_testing.git
   d16e92b..clcf2b8  main -> main
гілку "main" налаштовано на відстежування "origin/main".
oksanka@debian:~/Documents/automated_testing$
```

Посилання на GitHub:

https://github.com/OksanaMykytyn/automated_testing

Висновок: під час лабораторної роботи я навчилася будувати базове автоматизаційне рішення, яке включає кілька типів тестування. Я створила логічну структуру проєкту, у якій окремо розмістила unit, API, UI та E2E тести. Для сайту Видавництва Віват я реалізувала UI та E2E перевірки, а для API-тестів використала навчальний сервіс JSONPlaceholder. Я створила допоміжні функції для роботи з текстом і написала для них unit-тести. Під час роботи я застосувала принципи KISS та DRY, оскільки намагалася не ускладнювати структуру проєкту і винесла повторювану логіку в окремі функції. Я навчилася запускати всі тести разом, а також окремо запускати тільки unit, API, UI або E2E тести.